

Pandas for Machine Learning

These notes cover every pandas function you need for the practical component of your Machine Learning course — building the Naive Bayes, regression, decision tree, k-NN, SVM, ANN, and k-means/hierarchical clustering models listed in your syllabus. Each function includes its definition, syntax, common cases, a worked example, and a try-it-yourself exercise.

Why pandas, and not sklearn, in this document? Your syllabus practicals ask you to pull datasets from data.gov.in and the UCI ML repository, then split, clean, and evaluate models. Before any model (Naive Bayes, linear regression, k-NN, SVM, k-means...) ever sees your data, *pandas* is the tool that loads it, cleans it, selects features, scales it, and splits it. That is exactly the ground these notes cover — nothing here is generic pandas trivia; every function is tied to a specific unit in your syllabus.

Foundation

Pandas Basics — Series & DataFrame

Ties to: Unit 1 — "key elements" of a learning problem are almost always stored as a table of features and a target column. Every dataset you touch this semester (regression, classification, clustering) is loaded as a DataFrame first.

`pandas.Series(data, index=None, dtype=None, name=None)`

The 1-dimensional building block — think of it as a single labeled column

DEFINITION

A **Series** is a one-dimensional, labeled array that can hold any data type (numbers, strings, booleans). Every value has an index label attached to it, so it behaves like a cross between a Python list and a dictionary.

SYNTAX

```
pd.Series(data, index=None, dtype=None, name=None)
```

Parameter	Meaning
data	list, dict, ndarray, or scalar
index	labels for each value (defaults to 0,1,2,...)
dtype	force a data type, e.g. <code>float64</code>
name	name of the Series (becomes the column name if added to a DataFrame)

WHERE IT'S USED

Every single column you pull out of a DataFrame — `df['age']`, your target/label column `y` in regression or classification — is a Series.

EXAMPLE

```
import pandas as pd

marks = pd.Series([78, 65, 92, 55], index=['Amit', 'Bina', 'Chetan', 'Deepa'], name='Marks')
```

```
print(marks)
print(marks['Chetan'])
```

OUTPUT

```
Amit      78
Bina      65
Chetan    92
Deepa     55
Name: Marks, dtype: int64
92
```

COMMON CASES

- Creating a target vector: `y = df['species']`
- Creating from a dict — keys become the index automatically
- Arithmetic works element-wise: `marks * 1.1` curves every mark

► Try it — Exercise

`pandas.DataFrame(data, index=None, columns=None, dtype=None)`

The 2-dimensional table — your entire dataset

DEFINITION

A `DataFrame` is a 2-dimensional labeled table of rows and columns, like a spreadsheet or a SQL table. Columns can hold different data types (numeric, categorical, boolean).

SYNTAX

```
pd.DataFrame(data, index=None, columns=None, dtype=None)
```

WHERE IT'S USED

This is the object every algorithm in your syllabus consumes: rows = samples/observations, columns = features + a label column. Feature scaling, PCA, train/test splitting, decision trees, k-NN — all operate on a `DataFrame` (or the NumPy array beneath it).

EXAMPLE

```
import pandas as pd

data = {
    'age': [25, 32, 47, 51],
    'salary': [40000, 55000, 82000, 90000],
    'bought_insurance': [0, 0, 1, 1]
}
df = pd.DataFrame(data)
print(df)
print(df.shape)      # (rows, columns)
print(df.columns)     # column names
print(df.dtypes)      # data type per column
```

OUTPUT

```
   age  salary  bought_insurance
0   25   40000                 0
1   32   55000                 0
2   47   82000                 1
3   51   90000                 1
(4, 3)
Index(['age', 'salary', 'bought_insurance'], dtype='object')
age          int64
salary       int64
bought_insurance  int64
dtype: object
```

COMMON CASES

- From a dict of lists (columns), from a list of dicts (rows), or from a NumPy 2D array
- `df.shape`, `df.columns`, `df.index`, `df.dtypes` are attributes, not methods — no parentheses
- Splitting into features and target: `X = df.drop('bought_insurance', axis=1)`, `y = df['bought_insurance']` — this is literally the first line of every classification practical

► Try it — Exercise

Unit 1

Loading & Inspecting Data

Ties to: Unit 1 — "applications of ML" and the practicals instruction to use public datasets from data.gov.in and archive.ics.uci.edu/ml/datasets.php. This is always your first step, for every single practical this semester.

```
pandas.read_csv(filepath, sep=',', header='infer', index_col=None,
na_values=None)
```

Load a dataset from a CSV file into a DataFrame

DEFINITION

Reads a comma-separated (or other delimiter) text file and returns it as a DataFrame. This is how you will load nearly every UCI dataset (Iris, Wine, Breast Cancer, Boston Housing-style regression sets, etc.).

SYNTAX

```
pd.read_csv(filepath_or_buffer, sep=',', header='infer',
names=None, index_col=None, na_values=None, usecols=None)
```

Parameter	Meaning
<code>filepath_or_buffer</code>	path or URL to the file
<code>sep</code>	delimiter — use <code>'\t'</code> for tab-separated UCI files, <code>'+'</code> for some European datasets
<code>header</code>	row number to use as column names; <code>None</code> if the file has no header row (common in raw UCI <code>.data</code> files)

names	supply column names yourself when <code>header=None</code>
na_values	strings to treat as missing, e.g. <code>'?'</code> (UCI datasets often use <code>?</code> for missing values)
index_col	column to use as the row index

COMMON CASES

- **Standard CSV with header:** `pd.read_csv('iris.csv')`
- **Raw UCI .data file, no header, missing values marked '?':** `pd.read_csv('breast-cancer-wisconsin.data', header=None, na_values='?')`
- **Reading directly from a UCI URL:** pass the raw URL string instead of a local path
- **Tab-separated:** `pd.read_csv('data.tsv', sep='\t')`

EXAMPLE

```
import pandas as pd

# A typical UCI-style load: no header row, '?' marks missing values
cols = ['id', 'clump_thickness', 'cell_size', 'cell_shape', 'marginal_adhesion',
        'epithelial_size', 'bare_nuclei', 'bland_chromatin', 'normal_nucleoli',
        'mitoses', 'class']
df = pd.read_csv(
    'https://archive.ics.uci.edu/ml/machine-learning-databases/'
    'breast-cancer-wisconsin/breast-cancer-wisconsin.data',
    header=None, names=cols, na_values='?'
)
print(df.head(3))
```

OUTPUT

```
   id  clump_thickness  cell_size  ...  mitoses  class
0  1000025             5         1  ...        1      2
1  1002945             5         4  ...        1      2
2  1015425             3         1  ...        1      2
```

► Try it — Exercise

`df.head(n=5)` / `df.tail(n=5)`

Peek at the first / last rows

DEFINITION

`head(n)` returns the first n rows; `tail(n)` returns the last n . Default n is 5. The very first thing you should run after loading any dataset.

WHERE IT'S USED

Sanity-checking that a dataset loaded with correct columns, correct types, and no obvious misalignment before you do any preprocessing or feed it to a model.

EXAMPLE

```
print(df.head())      # first 5 rows
print(df.tail(3))     # last 3 rows
```

COMMON CASES

- `df.head()` with no argument → 5 rows
- `df.head(10)` → first 10 rows
- `df.sample(5)` is a good companion — a *random* 5 rows, useful when a dataset is sorted and `head()` alone would hide diversity

► Try it — Exercise

`df.info(verbose=True)`

A structural summary: columns, non-null counts, dtypes, memory

DEFINITION

Prints a concise summary of the DataFrame: number of rows, each column's name, count of non-null values, and dtype. This is the fastest way to spot missing values before you use `isnull()`.

SYNTAX

```
df.info()
```

EXAMPLE

```
df.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 699 entries, 0 to 698
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               699 non-null   int64
1   clump_thickness  699 non-null   int64
...
6   bare_nuclei      683 non-null   float64
10  class            699 non-null   int64
dtypes: float64(1), int64(10)
memory usage: 60.2 KB
```

COMMON CASES

- Column `bare_nuclei` shows 683 non-null out of 699 rows → 16 missing values → next step is `dropna()` or `fillna()` (Unit 2)
- A column you expected to be numeric shows dtype `object` → likely contains stray text or was read with a non-numeric placeholder

► Try it — Exercise

`df.describe(include=None)`

Summary statistics for every numeric column

DEFINITION

Computes count, mean, std, min, 25/50/75th percentiles, and max for every numeric column. With `include='all'` or `include='object'`, it also summarizes categorical columns (count, unique, top, freq).

WHERE IT'S USED

Directly relevant to Unit 2's feature scaling: `describe()` tells you the scale/range of each feature, which is exactly why algorithms like k-NN, k-means, and gradient descent-based regression need scaling — a feature ranging 0–90000 (salary) will dominate one ranging 0–1 (a ratio) unless scaled.

EXAMPLE

```
print(df.describe())
```

OUTPUT

	age	salary	bought_insurance
count	4.000000	4.000000	4.000000
mean	38.750000	66750.000000	0.500000
std	12.529970	23643.652974	0.577350
min	25.000000	40000.000000	0.000000
25%	30.250000	51250.000000	0.000000
50%	39.500000	68500.000000	0.500000
75%	48.000000	84000.000000	1.000000
max	51.000000	90000.000000	1.000000

COMMON CASES

- `df.describe()` — numeric columns only (default)
- `df.describe(include='object')` — categorical columns only
- `df.describe(include='all')` — everything, with NaN in cells that don't apply
- Comparing `min` / `max` across columns tells you immediately which features need scaling before k-NN, SVM, or k-means

► Try it — Exercise

Unit 2

Handling Missing & Duplicate Data

Ties to: Unit 2 — Preprocessing. Real UCI datasets (e.g. Breast Cancer Wisconsin) ship with missing values; models like linear/logistic regression, SVM, and k-NN cannot handle NaN directly.

`df.isnull()` / `df.notnull()`

Detect missing values, cell by cell

DEFINITION

`isnull()` returns a same-shaped DataFrame of booleans: `True` where a value is missing (NaN). `notnull()` is the exact opposite.

SYNTAX

```
df.isnull()           # full boolean grid
df.isnull().sum()     # missing count per column (most-used form)
df.isnull().sum().sum() # total missing values in the whole DataFrame
df['col'].isnull()    # boolean mask for one column
```

EXAMPLE

```
print(df.isnull().sum())
```

OUTPUT

```
id                0
clump_thickness   0
bare_nuclei       16
class             0
dtype: int64
```

COMMON CASES

- `df.isnull().sum()` — the standard first check, column-wise missing counts
- `df[df['bare_nuclei'].isnull()]` — view the actual rows that are missing that feature
- `df.isnull().any()` — which columns have *at least one* missing value (True/False per column)

► Try it — Exercise

`df.dropna(axis=0, how='any', subset=None, inplace=False)`

Remove rows (or columns) containing missing values

DEFINITION

Drops any row (default) or column that contains missing values, according to the `how` rule.

SYNTAX

```
df.dropna(axis=0, how='any', subset=None, inplace=False)
```

Parameter	Meaning
axis	0 = drop rows (default), 1 = drop columns
how	'any' = drop if any value missing; 'all' = drop only if every value in the row/col is missing

subset	only consider these columns when deciding to drop a row
inplace	if True, modifies df directly instead of returning a new one

EXAMPLE

```
print(df.shape)           # (699, 11)
df_clean = df.dropna()
print(df_clean.shape)     # (683, 11) -- 16 rows removed
```

COMMON CASES

- **Drop rows with any missing value:** `df.dropna()` — safe default for small amounts of missingness
- **Drop only if a specific column is missing:** `df.dropna(subset=['bare_nuclei'])`
- **Drop a column that is mostly empty:** `df.dropna(axis=1, thresh=len(df)*0.5)` — keeps only columns with at least 50% non-null data

When to prefer `dropna()` over `fillna()`: use it when missing values are rare (a small % of rows) so you don't lose much signal. If a large fraction of a column is missing, imputing with `fillna()` usually preserves more data for training your regression/classification model.

► Try it — Exercise

`df.fillna(value=None, method=None, inplace=False)`

Impute missing values instead of deleting rows

DEFINITION

Replaces NaN values with a specified value, a computed statistic, or by forward/backward filling from neighboring rows.

SYNTAX

```
df['col'].fillna(value, inplace=False)
df['col'].fillna(df['col'].mean())    # impute with the mean (regression-friendly)
df['col'].fillna(df['col'].median())  # impute with the median (robust to outliers)
df['col'].fillna(df['col'].mode()[0]) # impute with the mode (categorical columns)
df.fillna(method='ffill')            # forward-fill from the previous row
df.fillna(method='bfill')            # backward-fill from the next row
```

EXAMPLE

```
df['bare_nuclei'] = df['bare_nuclei'].fillna(df['bare_nuclei'].median())
print(df['bare_nuclei'].isnull().sum()) # 0
```

COMMON CASES

- **Numeric feature, no strong outliers** → fill with `mean()`
- **Numeric feature with outliers** (e.g. income) → fill with `median()`, which is not skewed by extreme values
- **Categorical feature** (e.g. a "yes/no" column) → fill with `mode()[0]`, the most frequent category
- **Time-ordered data** → `ffill` / `bfill` carries the last known value forward/backward

► Try it — Exercise

`df.duplicated(subset=None)` / `df.drop_duplicates(subset=None)`

Find and remove repeated rows

DEFINITION

`duplicated()` returns a boolean Series flagging rows that are exact repeats of an earlier row.

`drop_duplicates()` removes them, keeping the first occurrence by default.

SYNTAX

```
df.duplicated()           # boolean flag per row
df.duplicated(subset=['id']) # duplicate check on specific column(s) only
df.drop_duplicates(keep='first')
```

WHERE IT'S USED

Duplicate samples silently bias model training — a repeated row gets "double weight" during fitting and can leak between your train and test split if not removed first. Always check before splitting for Unit 3/4 practicals.

EXAMPLE

```
print(df.duplicated().sum()) # how many duplicate rows exist
df = df.drop_duplicates()
```

COMMON CASES

- Duplicate on all columns (default) — an exact repeated record
- Duplicate on a subset, e.g. `subset=['patient_id']`, to catch the same patient entered twice even if one field differs

► Try it — Exercise

Unit 2

Selection, Filtering & Feature Selection

Ties to: Unit 2 — "feature selection methods." Before PCA or any model, you need to pick out relevant columns/rows, drop irrelevant ones, and isolate features from the label.

`df.loc[row_labels, column_labels]`

Label-based selection

DEFINITION

Selects rows and columns by their *label* (index name / column name), not position. Supports slicing, boolean masks, and lists of labels. Note: with `loc`, slice endpoints are *inclusive*.

SYNTAX

```
df.loc[row_selector, column_selector]
df.loc[0:3, 'age']           # rows 0-3 inclusive, one column
df.loc[:, ['age','salary']]  # all rows, specific columns
df.loc[df['age'] > 40, 'salary'] # conditional selection
```

EXAMPLE

```
print(df.loc[0:2, ['age','salary']])
print(df.loc[df['age'] > 40, 'salary'])
```

OUTPUT

```
   age  salary
0   25   40000
1   32   55000
2   47   82000
2    82000
3    90000
Name: salary, dtype: int64
```

COMMON CASES

- Select all rows, a list of feature columns: `df.loc[:, ['sepal_length','sepal_width']]`
- Filter + select in one step: `df.loc[df['class']==1, 'age']`
- Set values conditionally: `df.loc[df['age']<0, 'age'] = 0` (clip invalid data)

► Try it — Exercise

`df.iloc[row_positions, column_positions]`

Position-based (integer index) selection

DEFINITION

Selects rows and columns by integer position, exactly like NumPy/Python list slicing. Slice endpoints are *exclusive* (unlike `loc`).

SYNTAX

```
df.iloc[row_positions, column_positions]
df.iloc[0:5, 0:3]      # first 5 rows, first 3 columns
df.iloc[:, :-1]        # all rows, all columns EXCEPT the last (common: features)
df.iloc[:, -1]         # all rows, ONLY the last column (common: target)
```

WHERE IT'S USED

This is the classic one-liner for splitting features from target when the target is the last column — extremely common with raw UCI files that have no column names:

EXAMPLE

```
X = df.iloc[:, :-1].values # every column except the last -> features
y = df.iloc[:, -1].values  # last column only -> target/label
print(X.shape, y.shape)
```

OUTPUT

```
(699, 10) (699,)
```

COMMON CASES

- `df.iloc[:, :-1]` — all features (drop last column)
- `df.iloc[:, -1]` — target column only
- `df.iloc[0:100]` — first 100 rows (e.g. a quick train subset)
- `df.iloc[-10:]` — last 10 rows

► Try it — Exercise

Boolean indexing: `df[condition]`

Filter rows using a logical condition

DEFINITION

Passing a boolean Series (produced by a comparison on a column) inside square brackets returns only the rows where the condition is `True`. Combine multiple conditions with `&` (and), `|` (or) — each condition must be wrapped in parentheses.

SYNTAX

```
df[df['col'] > value]
df[(df['col1'] > v1) & (df['col2'] == v2)]
df[df['col'].isin([v1, v2, v3])]
```

EXAMPLE

```
older_high_earners = df[(df['age'] > 30) & (df['salary'] > 50000)]
print(older_high_earners)
```

OUTPUT

```
   age  salary  bought_insurance
2   47   82000                  1
3   51   90000                  1
```

COMMON CASES

- Class-balance check before classification: `df[df['class']==1].shape[0]` vs. `df[df['class']==0].shape[0]`
- Removing invalid rows: `df = df[df['age'] > 0]`
- `isin()` for multi-value membership tests, useful for keeping only certain classes in a multi-class dataset

► Try it — Exercise

`df.drop(labels=None, axis=0, columns=None, inplace=False)`

Remove columns or rows

DEFINITION

Returns a new DataFrame with the specified row(s) or column(s) removed.

SYNTAX

```
df.drop('col_name', axis=1)           # drop one column
df.drop(columns=['c1', 'c2'])         # drop multiple columns (clearer syntax)
df.drop([0, 1, 2], axis=0)           # drop rows by index label
```

WHERE IT'S USED

The standard way to separate features (`X`) from the target (`y`) by name instead of position, and to remove irrelevant columns (e.g. an ID column) that would otherwise be treated — wrongly — as a predictive feature.

EXAMPLE

```
X = df.drop(columns=['id', 'class'])  # features only
y = df['class']                       # target only
print(X.columns.tolist())
```

COMMON CASES

- Dropping an ID column that has no predictive meaning before training any model
- Dropping the target column to build `X`
- `axis=1` (or `columns=`) for columns, `axis=0` (default) for rows

► Try it — Exercise

`df.select_dtypes(include=None, exclude=None)`

Grab only columns of a certain data type

DEFINITION

Returns a subset of the DataFrame containing only columns matching the given dtype(s). Useful for separating numeric features (which may need scaling / PCA) from categorical ones (which need encoding).

SYNTAX

```
df.select_dtypes(include=['number'])  # all numeric columns
df.select_dtypes(include=['object'])  # all text/categorical columns
```

EXAMPLE

```
numeric_cols = df.select_dtypes(include=['number']).columns
categorical_cols = df.select_dtypes(include=['object']).columns
print(numeric_cols.tolist())
print(categorical_cols.tolist())
```

COMMON CASES

- Automatically build a list of numeric columns to feed into feature scaling (Unit 2) without hardcoding names
- Automatically build a list of categorical columns to one-hot encode with `get_dummies()`

► Try it — Exercise

Unit 2 & 4

Scaling, Encoding & Transforming

Ties to: Unit 2 — "feature scaling" and dimensionality reduction (PCA needs scaled input); Unit 4 — classifiers like Naive Bayes and decision trees need categorical features encoded numerically.

`df.apply(func, axis=0)` / `Series.apply(func)`

Apply any custom function across rows, columns, or a single Series

DEFINITION

Runs a function along an axis of the DataFrame (`axis=0` = down each column, `axis=1` = across each row), or element-wise on a single Series. Accepts built-in functions, lambdas, or your own custom functions.

SYNTAX

```
df['col'].apply(func)           # element-wise on one column
df.apply(func, axis=0)         # func receives each COLUMN as a Series
df.apply(func, axis=1)         # func receives each ROW as a Series
```

EXAMPLE

```
# Feature engineering: turn continuous age into a category (Unit 4 prep)
def age_group(age):
    if age < 30: return 'young'
    elif age < 50: return 'middle'
    else: return 'senior'

df['age_group'] = df['age'].apply(age_group)

# Row-wise: compute a ratio feature from two columns
df['salary_per_age'] = df.apply(lambda row: row['salary'] / row['age'], axis=1)
print(df[['age', 'age_group', 'salary_per_age']])
```

COMMON CASES

- Single-column transform: `df['col'].apply(lambda x: x**2)`
- Row-wise feature engineering combining multiple columns: `axis=1`

- Binning a continuous variable into categories before classification

► Try it — Exercise

Series.map(arg)

Element-wise substitution using a function, dict, or Series

DEFINITION

Substitutes each value in a Series according to a mapping — a function, a dictionary of old→new values, or another Series. Simpler and often faster than `apply()` for pure lookup-style replacement, and works only on a Series (not a whole DataFrame).

SYNTAX

```
df['col'].map({'old1':'new1', 'old2':'new2'})
df['col'].map(lambda x: x.upper())
```

WHERE IT'S USED

The classic way to manually encode a binary/ordinal categorical target before Naive Bayes, logistic regression, decision trees, etc., when you want explicit control over which label maps to which number (unlike the automatic alphabetical mapping `get_dummies()` would apply).

EXAMPLE

```
df['class'] = df['class'].map({2: 0, 4: 1}) # UCI encodes benign=2, malignant=4
print(df['class'].value_counts())
```

OUTPUT

```
0    458
1    241
Name: class, dtype: int64
```

COMMON CASES

- Recoding a target's raw class labels into clean 0/1 for binary classification
- Mapping abbreviations to full names for readability, e.g. `{'M':'Male','F':'Female'}`

► Try it — Exercise

Manual feature scaling with pandas (Min-Max & Standardization)

The pandas-only way to scale features — no sklearn required

DEFINITION

Feature scaling brings all numeric columns to a comparable range. Two formulas matter for this syllabus (Unit 2, 6.1.1/6.1.2):

- **Min-Max normalization** → squeezes values into [0, 1]: $(x - \min) / (\max - \min)$
- **Standardization (Z-score)** → mean 0, std 1: $(x - \text{mean}) / \text{std}$

SYNTAX

```
# Min-Max scaling, column by column
df_norm = (df - df.min()) / (df.max() - df.min())

# Standardization (z-score)
df_std = (df - df.mean()) / df.std()
```

EXAMPLE

```
features = ['age', 'salary']

# Min-max
df_norm = df[features].copy()
df_norm = (df_norm - df_norm.min()) / (df_norm.max() - df_norm.min())
print(df_norm)

# Standardization
df_std = df[features].copy()
df_std = (df_std - df_std.mean()) / df_std.std()
print(df_std)
```

OUTPUT (MIN-MAX, FOR THE EARLIER AGE/SALARY EXAMPLE)

```
   age  salary
0  0.000000  0.000000
1  0.269231  0.300000
2  0.846154  0.840000
3  1.000000  1.000000
```

COMMON CASES

- **Use Min-Max** when you need bounded [0,1] values, e.g. before k-NN or k-means, where raw distance calculations are sensitive to feature range
- **Use Standardization** before algorithms that assume roughly normal, zero-centered data — SVM, logistic regression, PCA (Unit 2, 12.2), and gradient-descent-based linear regression (Unit 3) converge faster and more reliably on standardized features
- Always fit scaling parameters (min/max or mean/std) on the *training set only*, then apply the same parameters to the test set — never recompute them on test data (this avoids "data leakage")

► Try it — Exercise

`pandas.get_dummies(data, columns=None, drop_first=False)`

One-hot encode categorical columns

DEFINITION

Converts a categorical column into multiple binary (0/1) columns, one per category. Required because most ML algorithms in this syllabus (SVM, logistic regression, k-NN, neural networks) need purely numeric input.

SYNTAX

```
pd.get_dummies(df, columns=['species'], drop_first=False)
```

Parameter	Meaning
columns	which categorical column(s) to encode
drop_first	drop one dummy column per feature to avoid redundant/collinear columns (recommended for linear/logistic regression)

EXAMPLE

```
iris_encoded = pd.get_dummies(iris_df, columns=['species'], drop_first=True)
print(iris_encoded.columns.tolist())
```

OUTPUT

```
['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
 'species_Iris-versicolor', 'species_Iris-virginica']
```

COMMON CASES

- **Nominal categories with no order** (color, city, species) → one-hot encode with `get_dummies()`
- **drop_first=True** for linear/logistic regression, to avoid the "dummy variable trap" (perfectly correlated columns)
- **Ordinal categories with a real order** (low/medium/high) → prefer `.map()` to assign 0/1/2 rather than one-hot encoding, since order carries meaning

► Try it — Exercise

df.replace(to_replace, value)

Substitute specific values anywhere in the DataFrame

DEFINITION

Replaces occurrences of a value (or list of values) with another value, across a Series or the whole DataFrame. More flexible than `map()` because it can target values without replacing everything else with NaN.

SYNTAX

```
df.replace('?', np.nan) # normalize a missing-value marker
df['col'].replace([2, 4], [0, 1]) # replace a list of values with another list
df.replace({'col1': {2:0, 4:1}, 'col2': {}}) # per-column replacement dict
```

EXAMPLE

```
import numpy as np
df = df.replace('?', np.nan) # standardize UCI's missing marker before isnull()/dropna()
df['bare_nuclei'] = df['bare_nuclei'].astype(float)
```


COMMON CASES

- Normalizing inconsistent missing-value markers (`'?'`, `'NA'`, `-1`) into a real `NaN` so `isnull()` / `dropna()` / `fillna()` can find them
- Fixing inconsistent category spellings, e.g. `{ 'Male': 'M', 'male': 'M' }`

► Try it — Exercise

`df.astype(dtype)`

Convert a column's data type

DEFINITION

Casts a Series or DataFrame to a specified dtype, e.g. `object` → `float64`, or `int` → `category`.

SYNTAX

```
df['col'] = df['col'].astype('float64')
df['col'] = df['col'].astype('category')
df['col'] = df['col'].astype(int)
```

WHERE IT'S USED

After `read_csv` reads a numeric column as `object` (usually because of a stray non-numeric marker like `'?'`), you must cast it back to numeric before scaling, correlation, or model training — none of those work on text-typed columns.

EXAMPLE

```
print(df['bare_nuclei'].dtype)          # object (because of '?' before cleaning)
df['bare_nuclei'] = df['bare_nuclei'].astype(float)
print(df['bare_nuclei'].dtype)          # float64
```

COMMON CASES

- Numeric column stuck as `object` after loading → `.astype(float)`
- Converting a low-cardinality string column to `'category'` dtype to save memory before encoding

► Try it — Exercise

UNIT 2 — COMBINED PREPROCESSING EXERCISE

1. Load the Breast Cancer Wisconsin dataset from UCI with `read_csv`, treating `'?'` as missing.
2. Use `info()` and `isnull().sum()` to find which column has missing values.
3. Impute that column with its median using `fillna()`.
4. Drop the `id` column with `drop()` — it holds no predictive value.
5. Recode the target column (2/4) to (0/1) using `.map()`.
6. Standardize all remaining feature columns using the z-score formula from the scaling section.

7. Confirm with `describe()` that every feature now has mean ≈ 0 and std ≈ 1 .

Unit 3 & 4

Splitting, Sampling & Combining Datasets

Ties to: the practicals instruction to "split datasets into training and test sets" for every regression/classification model, and to "perform k-cross-validation."

```
df.sample(n=None, frac=None, random_state=None)
```

Randomly sample rows — the pandas-only way to train/test split

DEFINITION

Returns a random subset of rows. Combined with `drop()` on the sampled index, this gives you a manual train/test split using only pandas (useful if you want to understand the mechanics before relying on

`sklearn.model_selection.train_test_split`).

SYNTAX

```
df.sample(n=50)                # exactly 50 random rows
df.sample(frac=0.8, random_state=42) # 80% of rows, reproducible with a fixed seed
```

EXAMPLE

```
# Manual 80/20 train-test split using only pandas
train = df.sample(frac=0.8, random_state=42)
test = df.drop(train.index)

print(train.shape, test.shape)
```

OUTPUT

```
(559, 10) (140, 10)
```

COMMON CASES

- `random_state` fixes the random seed so your split is reproducible across runs — important when comparing model runs for your practical report
- `frac=1` is a common trick to *shuffle* an entire DataFrame (e.g. before k-fold cross-validation, since some datasets are sorted by class)

► Try it — Exercise

```
pandas.concat(objs, axis=0, ignore_index=False)
```

Stack DataFrames together, vertically or side by side

DEFINITION

Concatenates a list of DataFrames along an axis: `axis=0` stacks rows on top of each other (same columns), `axis=1` places them side by side (same rows, different columns).

SYNTAX

```
pd.concat([df1, df2], axis=0, ignore_index=True) # stack rows
pd.concat([df1, df2], axis=1)                  # join columns side-by-side
```

WHERE IT'S USED

Recombining a scaled feature set with an untouched target column after processing them separately, or merging results from k-fold cross-validation splits back together for reporting.

EXAMPLE

```
X_scaled = (X - X.mean()) / X.std() # scaled features
final_df = pd.concat([X_scaled, y], axis=1) # reattach the target column
print(final_df.head())
```

COMMON CASES

- `axis=0` — combining multiple CSV files of the same structure (e.g. multiple months of a dataset) into one DataFrame
- `axis=1` — reattaching a target column after scaling only the feature columns
- `ignore_index=True` — re-numbers the index cleanly after stacking rows, avoiding duplicate index labels

► Try it — Exercise

`pandas.merge(left, right, on=None, how='inner')`

SQL-style join of two DataFrames on a common key

DEFINITION

Combines two DataFrames based on matching values in one or more shared columns (a "key"), similar to a SQL JOIN.

SYNTAX

```
pd.merge(left, right, on='key_column', how='inner')
```

how	Meaning
'inner'	keep only keys present in <i>both</i> DataFrames (default)
'left'	keep all rows from the left DataFrame, matching where possible
'right'	keep all rows from the right DataFrame
'outer'	keep all rows from both, filling gaps with NaN

EXAMPLE

```
patients = pd.DataFrame({'patient_id':[1,2,3], 'age':[45,60,32]})
diagnosis = pd.DataFrame({'patient_id':[1,2,3], 'class':[0,1,0]})
```

```
full = pd.merge(patients, diagnosis, on='patient_id', how='inner')
print(full)
```

OUTPUT

```
  patient_id  age  class
0          1   45      0
1          2   60      1
2          3   32      0
```

COMMON CASES

- Joining a feature table and a label table that were provided as two separate files sharing an ID column — common with government open datasets on data.gov.in
- `how='left'` when you want to keep every record from your main table even if some auxiliary info is missing

► Try it — Exercise

Unit 3 & 5

Exploring Relationships — Sorting, Grouping, Correlation

Ties to: Unit 3 — understanding which features relate to the regression target; Unit 5 — exploring natural groupings before/after clustering; and general class-balance checks for Unit 4.

`df.sort_values(by, ascending=True)`

Sort rows by one or more column values

DEFINITION

Reorders the DataFrame's rows according to the values in one or more columns.

SYNTAX

```
df.sort_values(by='salary', ascending=False)
df.sort_values(by=['class', 'age'], ascending=[True, False])
```

EXAMPLE

```
print(df.sort_values(by='salary', ascending=False).head(3))
```

COMMON CASES

- Ranking rows by a regression target to spot outliers at the extremes
- Sorting a correlation result (see `corr()` below) to find the strongest predictors of a target quickly

► Try it — Exercise

df.groupby(by)

Split-apply-combine: aggregate statistics per category

DEFINITION

Splits the DataFrame into groups based on unique values of one or more columns, applies an aggregation function to each group, then combines the results into a new DataFrame/Series. The engine behind almost all category-wise summary statistics.

SYNTAX

```
df.groupby('species').mean()
df.groupby('species')['petal_length'].mean()
df.groupby(['species', 'class']).agg(['mean', 'std', 'count'])
```

WHERE IT'S USED

Directly useful for Unit 5 (clustering) — comparing per-cluster feature averages after you assign k-means labels back onto the DataFrame — and for Unit 4, checking whether classes are balanced before training a classifier.

EXAMPLE

```
print(iris_df.groupby('species').mean())
```

OUTPUT

	sepal_length	sepal_width	petal_length	petal_width
species				
Iris-setosa	5.006	3.418	1.464	0.244
Iris-versicolor	5.936	2.770	4.260	1.326
Iris-virginica	6.588	2.974	5.552	2.026

COMMON CASES

- Mean/std of each feature per class → sanity-check that classes are actually separable before training a classifier
- After running k-means (Unit 5), assign the resulting cluster label as a new column, then `groupby('cluster').mean()` to interpret what each cluster represents
- `df.groupby('class').size()` — a quick class-balance check, equivalent to `value_counts()` on the target

► Try it — Exercise

df.corr(method='pearson')

Pairwise correlation between all numeric columns

DEFINITION

Computes the pairwise correlation coefficient (Pearson by default, range -1 to 1) between every pair of numeric columns, returned as a square matrix.

SYNTAX

```
df.corr()                # Pearson correlation matrix
df.corr()['target'].sort_values(ascending=False)  # correlation with just one column, ranked
```

WHERE IT'S USED

Direct support for Unit 2's "feature selection methods" and Unit 3's regression: features weakly correlated with the target contribute little predictive value and are candidates for removal; features very highly correlated with *each other* (multicollinearity) can destabilize linear regression coefficients.

EXAMPLE

```
print(df.corr()['salary'].sort_values(ascending=False))
```

OUTPUT

```
salary          1.000000
age             0.987421
bought_insurance 0.958394
Name: salary, dtype: float64
```

COMMON CASES

- Ranking every feature's correlation with the regression target to guide manual feature selection before Unit 3 models
- Spotting two features correlated near ± 1 with each other — a sign to drop one before linear regression
- Visualizing the same matrix as a heatmap (with matplotlib/seaborn) is a common companion step, though outside pandas itself

► Try it — Exercise

Series.value_counts(normalize=False)

Frequency count of each unique value

DEFINITION

Returns the count of each unique value in a Series, sorted descending by default. With `normalize=True`, returns proportions instead of raw counts.

SYNTAX

```
df['class'].value_counts()
df['class'].value_counts(normalize=True)  # as percentages (0-1)
```

WHERE IT'S USED

The standard first check for class imbalance before training any classifier (Unit 4) — a 90/10 class split needs different handling (e.g. stratified sampling, different evaluation metrics) than a balanced 50/50 split.

EXAMPLE

```
print(df['class'].value_counts(normalize=True))
```

OUTPUT

```
0    0.655
1    0.345
Name: class, dtype: float64
```

COMMON CASES

- Checking class balance before choosing evaluation metrics — with imbalance, accuracy alone is misleading and metrics like Recall, F1-score, and AUC (as your syllabus specifies) matter more
- Counting categories in a categorical feature before deciding whether to one-hot encode (few categories) or group rare ones together (many categories)

► Try it — Exercise

Capstone Exercise — Full Preprocessing Pipeline

Combine everything above into one end-to-end pipeline, exactly as you'll need to for your practical file. Pick **any one** of these UCI datasets: Breast Cancer Wisconsin (classification), Wine Quality (regression), or Iris (classification/clustering).

STEPS

1. **Load** the dataset with `read_csv()`, handling missing-value markers with `na_values`.
2. **Inspect** with `head()`, `info()`, and `describe()`.
3. **Clean**: check `isnull().sum()`; impute or drop as appropriate with `fillna()` / `dropna()`. Remove duplicates with `drop_duplicates()`.
4. **Select features**: drop irrelevant columns (IDs) with `drop()`; use `corr()` to identify weakly-related features to consider removing.
5. **Encode** any categorical columns with `get_dummies()` or `.map()`.
6. **Scale** numeric features with the standardization formula.
7. **Check balance** of the target with `value_counts(normalize=True)` (classification) — or check its distribution with `describe()` (regression).
8. **Split** into train/test with `sample()` + `drop()` (or hand off `X / y` built with `iloc` to `sklearn.model_selection.train_test_split` for your actual practical).
9. **Hand off** the cleaned, scaled `X_train, X_test, y_train, y_test` to whichever model your practical requires — Naive Bayes, linear/logistic regression, k-NN, decision tree, SVM, or k-means — this is where pandas' job ends and the model-fitting begins.

```
import pandas as pd
import numpy as np

# 1. Load
df = pd.read_csv('dataset.csv', na_values='?')
```

```
# 2. Inspect
print(df.info())
print(df.describe())

# 3. Clean
df = df.drop_duplicates()
df['col_with_gaps'] = df['col_with_gaps'].fillna(df['col_with_gaps'].median())

# 4. Select features
df = df.drop(columns=['id'])

# 5. Encode (if a categorical column exists)
df = pd.get_dummies(df, columns=['categorical_col'], drop_first=True)

# 6. Scale
target = df['target']
features = df.drop(columns=['target'])
features_scaled = (features - features.mean()) / features.std()

# 7. Check target balance / distribution
print(target.value_counts(normalize=True))

# 8. Split
train_idx = features_scaled.sample(frac=0.8, random_state=42).index
X_train, X_test = features_scaled.loc[train_idx], features_scaled.drop(train_idx)
y_train, y_test = target.loc[train_idx], target.drop(train_idx)

print(X_train.shape, X_test.shape)
```